Step 1 — Connect to One MCP Server and List Its Tools — Solution

> Back to the step · Hint 1 · Hint 2

Story: prove the smallest real piece before building on it — a client that launches a server over stdio, does the MCP handshake, and lists what the server offers. No agent, no model. **If not:** every later bug would be two bugs at once — a broken loop *and* a broken connection — and you'd never know which.

Every file lives under the repo folder. `main.py` runs from the repo root. All of this code runs against the example server with no API key.

`src/exceptions.py`

```
# The project's named errors. One per real failure mode, so callers can
# catch exactly what they mean to handle.
class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass


class ServerUnreachableError(Exception):
    """Raised when an MCP server cannot be connected to in time."""
    pass


class MaxIterationsExceeded(Exception):
    """Raised when the ReAct loop runs more steps than its hard limit."""
    pass
```

`src/config.py`

```
# Loads settings from .env into a checked Config object (the Doc01 pattern).
# load_config() raises MissingConfigError when OPENAI_API_KEY is missing.
import os
from dataclasses import dataclass
```

```
from dotenv import load_dotenv
from src.exceptions import MissingConfigError


@dataclass
class Config:
    openai_api_key: str
    log_level: str
    connect_timeout: float
    max_iterations: int


def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value


def load_config() -> Config:
    load_dotenv()
    api_key = require_env("OPENAI_API_KEY")
    log_level = os.getenv("LOG_LEVEL", "INFO")
    connect_timeout = float(os.getenv("MCP_CONNECT_TIMEOUT", "10"))
    max_iterations = int(os.getenv("MCP_MAX_ITERATIONS", "6"))
    return Config(
        openai_api_key=api_key,
        log_level=log_level,
        connect_timeout=connect_timeout,
        max_iterations=max_iterations,
    )
```

src/utils/logger.py

```
# One logging setup for every file. Logs to STDERR on purpose: an MCP
# client can carry protocol traffic on stdout, and a stray print there
# would corrupt it.
import logging
import os
import sys


def get_logger(name: str) -> logging.Logger:
```

```
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler(sys.stderr)
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

`servers/notes_server.py`

Story: a tiny real MCP server so the whole project runs with no external setup — a `MCPServer`, two `@mcp.tool()`s, `mcp.run(transport="stdio")`. The docstring is the tool's description, which the client shows the model. **If not:** you'd need Node.js and a public server just to test Step 1.

```
# A minimal MCP server (notes). The client launches this as a subprocess
# and discovers its tools — it never imports anything from this file.
# All logging goes to stderr; stdout is the MCP protocol channel.
import sys
from mcp.server.mcpserver import MCPServer

mcp = MCPServer("notes-server")

NOTES = {
    "n1": "Q3 roadmap: ship the billing API, then the mobile app beta.",
    "n2": "Deploy process: run deploy.sh after the staging check passes.",
}


@mcp.tool()
def search_notes(query: str) -> str:
    """Search notes by a keyword; return matching note ids."""
    hits = []
    for note_id, text in NOTES.items():
        if query.lower() in text.lower():
            hits.append(f"{note_id}: {text[:30]}")
    if not hits:
        return "No matching notes."
    return "\n".join(hits)


@mcp.tool()
```

```
def read_note(note_id: str) -> str:
    """Read the full text of one note by its id."""
    if note_id not in NOTES:
        return f"Error: no note with id {note_id}"
    return NOTES[note_id]


if __name__ == "__main__":
    print("notes server starting", file=sys.stderr)
    mcp.run(transport="stdio")
```

`src/servers_config.py`

```
# The servers to connect to, as plain data. The agent never hardcodes
# these servers' TOOLS — only how to launch them. One server for now;
# Step 3 adds a second.
SERVERS = [
    {"name": "notes", "command": "python",
     "args": ["servers/notes_server.py"]},
]
```

`src/server_pool.py` (Step 1 version)

Story: one place owns every server connection. `connect_all()` launches each server over stdio, does the `initialize()` handshake, lists its tools, and stores each one under a namespaced name `servername__toolname` with a routing entry back to its real server + tool. **If not:** connection code would be scattered through the agent, and there'd be nowhere clean to add multi-server routing (Step 3) or resilience (Step 4).

```
# MCPServerPool: connect to every configured server, discover each one's
# tools, and namespace them. The heart of the project. (Step 1: connect +
# discover only; call_tool/openai_tools arrive in Step 2, resilience in
# Step 4.)
from contextlib import AsyncExitStack

from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

from src.utils.logger import get_logger

NAME_SEPARATOR = "__"
```

```
class MCPServerPool:
    def __init__(self, servers, connect_timeout=10.0):
        self.servers = servers
        self.connect_timeout = connect_timeout
        self.stack = AsyncExitStack()
        self.sessions = {}
        self.routing = {}
        self.named_tools = []
        self.logger = get_logger(__name__)

    async def _connect_one(self, server):
        params = StdioServerParameters(
            command=server["command"], args=server["args"]
        )
        read, write = await self.stack.enter_async_context(
            stdio_client(params)
        )
        session = await self.stack.enter_async_context(
            ClientSession(read, write)
        )
        await session.initialize()
        return session

    async def connect_all(self):
        for server in self.servers:
            name = server["name"]
            session = await self._connect_one(server)
            self.sessions[name] = session
            listed = await session.list_tools()
            for tool in listed.tools:
                namespaced = name + NAME_SEPARATOR + tool.name
                self.routing[namespaced] = (name, tool.name)
                self.named_tools.append((namespaced, tool))
            self.logger.info(f"[{name}] connected, {len(listed.tools)} tools")

    async def aclose(self):
        await self.stack.aclose()
```

`main.py` (Step 1 version)

Story: connect, and print exactly what the model will later be handed. Looking at the real `input_schema` now — not what you assume it looks like — is the whole point of Step 1. **If not:** Step 2's schema bridge would be guessing at a shape you never checked.

```
# Step 1 demo: connect to the configured servers and print every
# discovered tool, so you see the real shape before any agent uses it.
import asyncio
import sys

from src.config import load_config
from src.exceptions import MissingConfigError
from src.server_pool import MCPServerPool
from src.servers_config import SERVERS


async def run_demo():
    try:
        load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    pool = MCPServerPool(SERVERS)
    await pool.connect_all()
    try:
        print(f"Connected servers: {list(pool.sessions.keys())}")
        for namespaced, tool in pool.named_tools:
            properties = tool.input_schema.get("properties", {})
            print(f"\nTool: {namespaced}")
            print(f"  description: {tool.description}")
            print(f"  inputs: {list(properties.keys())}")
    finally:
        await pool.aclose()


def main():
    asyncio.run(run_demo())


if __name__ == "__main__":
    main()
```

Expected output (`python main.py` , with a valid `.env`):

```
Connected servers: ['notes']

Tool: notes__search_notes
  description: Search notes by a keyword; return matching note ids.
  inputs: ['query']
```

```
Tool: notes__read_note
  description: Read the full text of one note by its id.
  inputs: ['note_id']
```

> **A real gotcha, confirmed by running it:** in the current `mcp` SDK a discovered tool's schema is on `tool.input_schema` (snake_case). The attribute is **not** `tool.inputSchema` — that raises `AttributeError: 'Tool' object has no attribute 'inputSchema'`. (The JSON wire format still uses the `inputSchema` key, but the Python attribute is `input_schema`.) Here we read `tool.input_schema["properties"]` to print just the parameter names; Step 2's bridge passes the whole `input_schema` straight through to OpenAI.

`tests/unit/test_config.py`

```
# Checks the settings code with no network. Each case prints OK or WRONG.
# It changes only a made-up setting, so your real .env is never touched.
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "MCPBRIDGE_TEST_SETTING"


def test_missing_setting() -> None:
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")


def test_empty_setting() -> None:
    os.environ[TEST_SETTING] = ""
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 2 (empty setting)   -> {result}")
```

```
def test_config_loads() -> None:
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    if config.max_iterations <= 0:
        result = "WRONG"
    print(f"case 3 (config loads)    -> {result}")


if __name__ == "__main__":
    test_missing_setting()
    test_empty_setting()
    test_config_loads()
```

Expected output (`python -m tests.unit.test_config`):

```
case 1 (missing setting) -> OK
case 2 (empty setting)   -> OK
case 3 (config loads)    -> OK
```

Run `main.py` once more against a deliberately wrong command (e.g. a server path that doesn't exist) and confirm you get a clear exception, not a silent hang — that's the failure Step 4 will catch and skip gracefully.